Building an Automated Avatar Generation Service

Feasibility Analysis & Claude Code Development Plan

March 15, 2026

The Big Idea

Build a web service that takes a text prompt or image and outputs a complete, TalkingHead-ready GLB avatar — with split lips, teeth, tongue, eyes, 52 ARKit blend shapes, 15 Oculus visemes, a Mixamo-compatible rig, and animations — all automatically. Essentially: the child of Ready Player Me and Tripo AI, but fully automated and AI-generated.

Why This Is Now Feasible (It Wasn't 12 Months Ago)

Five open-source breakthroughs from 2025 have collectively cracked every stage of this pipeline:

1. TRELLIS.2 (Microsoft, Dec 2025) — MIT License

- 4B parameter image-to-3D model
- Generates meshes with PBR materials (base color, roughness, metallic, opacity)
- Handles open surfaces, non-manifold geometry, transparency
- 3 seconds at 512^3, 17 seconds at 1024^3 on H100
- GLB export built in
- **Can run on 24GB+ VRAM (RTX 4090/5090)**
- Full training code released — can be fine-tuned on humanoid datasets

2. Hunyuan3D 2.1/2.5 (Tencent, Jan–Jun 2025) — Tencent License

- Text-to-3D and image-to-3D with PBR textures
- Shape generation on 6GB VRAM (mini/turbo models)
- Blender addon available
- ComfyUI integration
- **Hunyuan3D Studio paper describes a humanoid auto-rigging module with 22 joints and motion retargeting**

3. UniRig (VAST-AI/Tripo + Tsinghua, SIGGRAPH 2025) — MIT License

- Automatic skeleton prediction + skinning weights for ANY 3D mesh
- GPT-based autoregressive skeleton generation
- 215% improvement over previous methods in rigging accuracy
- Trained on 14,000+ rigged models (Rig-XL dataset)
- **Predicts Mixamo-compatible bone structures** including spring bones for hair/cloth
- 1–5 seconds inference time
- CLI-based: `generate_skeleton.sh --input model.glb --output skeleton.fbx`

4. Deformation Transfer for ARKit Blend Shapes (open source implementations exist)

- The Sumner et al. deformation transfer algorithm can automatically transfer all 52 ARKit expressions from a reference mesh to any target mesh with different topology
- Open source Python implementation: github.com/vasiliskatr/deformation_transfer_ARkit_blendshapes
- Requires: source mesh with all 52 blend shapes (can use ICT FaceKit, which is free) + correspondence mapping
- This is THE key to automating the hardest step

5. Headless Blender as a Pipeline Engine

- Blender runs fully headless via `blender --background --python script.py`
- All mesh operations (separation, UV unwrapping, retopology, shape key creation, GLB export) are scriptable via bpy
- Can be containerized in Docker for server deployment
- Blender 4.x has excellent GLTF/GLB export with morph target support

---

The Architecture: Template-Hybrid Approach

The critical insight is: **don't try to post-process arbitrary AI-generated meshes into TalkingHead format. Instead, use a parameterized template mesh system where AI controls the variation.**

Why Template-Hybrid Beats Pure AI Generation

Pure AI-generated meshes (from TRELLIS.2, Hunyuan3D, etc.) produce a single fused mesh blob. Converting that into separate eyes, teeth, tongue, etc. with proper topology for 67 blend shapes is effectively impossible to automate reliably.

Instead, the architecture works like this:

```
User prompt: "A cyberpunk female warrior with blue hair and facial scars"
                    |
                    ▼
    ┌──────────────────────────────┐
    |  Stage 1: Concept Generation |  Stable Diffusion / FLUX
    |  Text → Reference Image(s)   |  Generate front + side views
    └──────────────────────────────┘
                    |
                    ▼
    ┌──────────────────────────────┐
    |  Stage 2: Base Mesh Selection |  Parameterized template library
    |  Pick/blend body type, face   |  Pre-built with split geometry,
    |  shape, proportions           |  proper topology, vertex groups
    └──────────────────────────────┘
                    |
                    ▼
    ┌──────────────────────────────┐
    |  Stage 3: Appearance Transfer |  TRELLIS.2 or Hunyuan3D-Paint
    |  Generate texture/appearance  |  for the template mesh, guided
    |  from the concept images      |  by reference images
    └──────────────────────────────┘
                    |
                    ▼
    ┌──────────────────────────────┐
    |  Stage 4: Shape Deformation   |  Deformation transfer from
    |  Transfer face expressions    |  ICT FaceKit reference →
    |  52 ARKit + 15 Oculus visemes |  target template mesh
    └──────────────────────────────┘
                    |
                    ▼
    ┌──────────────────────────────┐
    |  Stage 5: Auto-Rig            |  UniRig or template rig
    |  Skeleton + skinning weights  |  (template already has rig,
    |  Mixamo-compatible bones      |  just needs weight transfer)
    └──────────────────────────────┘
                    |
                    ▼
    ┌──────────────────────────────┐
    |  Stage 6: Animation + Export  |  Mixamo FBX animations
    |  Attach standard anims        |  Export as GLB with all
    |  Export TalkingHead-ready GLB |  morph targets
    └──────────────────────────────┘
```

Alternative: Pure AI Generation Path (Higher Risk, Higher Novelty)

For users who want truly unique body shapes that templates can't cover:

```
User prompt → TRELLIS.2/Hunyuan3D → raw mesh
    → Blender headless: auto-retopology (Instant Meshes or built-in)
    → Blender headless: face landmark detection (MediaPipe on rendered views)
    → Blender headless: separate eye sockets, model teeth/tongue from prefab
library
    → UniRig: auto-rig skeleton + skinning
    → Deformation transfer: apply ARKit blend shapes from nearest template
    → Export GLB
```

This path is less reliable but produces more varied results. Both paths should be offered.

Claude Code Development Phases

Phase 0: Foundation & Research Spike (1–2 weeks)

Goal: Prove each pipeline stage works independently on your machine.

```
Tasks for Claude Code:
- Set up a Python project with virtual environment
- Install Blender 4.x headless + bpy scripting environment
- Install TRELLIS.2 locally (or Hunyuan3D-2mini for 16GB VRAM)
- Install UniRig from VAST-AI/UniRig repo
- Download ICT FaceKit reference mesh (free, public)
- Clone deformation_transfer_ARkit_blendshapes repo
- Write a smoke test that:
  1. Generates a 3D mesh from a test image via TRELLIS.2
  2. Runs UniRig skeleton prediction on the mesh
  3. Runs a simple deformation transfer test
  4. Exports a GLB from Blender headless
```

Deliverable: Each component runs independently. You understand the input/output formats.

Phase 1: Template Mesh System (2–3 weeks)

Goal: Create a parameterized template mesh library with all TalkingHead requirements.

```
Tasks for Claude Code:
- Create 3-5 base body templates in Blender:
  - Male realistic, female realistic, androgynous
  - Stylized/cartoon male, stylized/cartoon female
  - Each with: split lips, upper teeth, lower teeth, tongue,
    separate eye meshes, proper face topology (~2000 faces for head)
  - Each already UV-unwrapped with clean UV islands
  - Total mesh: ~8000-10000 faces each
  - Vertex groups labeled for all face regions

- Write Blender Python scripts for parametric variation:
  - Body proportions (height, shoulder width, hip width, etc.)
  - Face shape blending (round, angular, long, wide, etc.)
  - Eye size/spacing/shape
  - Nose shape variants
  - Mouth size/shape
  - These use shape keys on the template to blend between variants

- Pre-rig each template:
  - Mixamo-compatible skeleton (already placed)
  - Skinning weights already painted
  - This avoids needing UniRig for the template path

- Create the 52 ARKit blend shapes for each template:
  - Use deformation transfer from ICT FaceKit
  - Automate via Blender headless script
  - Validate each shape key works

- Create the 15 Oculus viseme shapes:
  - Derive from ARKit mouth shapes
  - Use TalkingHead's conversion script as reference

- Write validation script:
  - Export template as GLB
  - Load in TalkingHead test page
  - Verify all blend shapes animate correctly
```

Deliverable: 3–5 template GLBs that work perfectly in TalkingHead. A parametric Blender script that can generate variations.

Phase 2: AI Appearance Pipeline (2–3 weeks)

Goal: Given a text prompt or image, generate unique textures for template meshes.

```
Tasks for Claude Code:
- Integrate Stable Diffusion / FLUX for concept art generation:
  - Text → front-view character portrait
  - Text → character turnaround sheet (front/side/back)
  - Use ControlNet or IP-Adapter for consistency

- Integrate texture generation:
  - Option A: Hunyuan3D-Paint (texture existing meshes from reference)
  - Option B: TEXTure or similar UV-space painting from views
  - Option C: Custom projection mapping (render template from
    multiple views, inpaint/transfer style, project back to UV)

- Build the prompt processing pipeline:
  - Parse user prompt into structured attributes:
    - Gender/body type → template selection
    - Style (realistic/cartoon/anime) → template selection
    - Features (hair color, skin tone, clothing) → texture gen
    - Special features (scars, tattoos, etc.) → texture gen
  - Use Claude API or local LLM for prompt decomposition

- Hair system:
  - Option A: Hair cards (textured planes) — common for games
  - Option B: Particle hair → mesh conversion in Blender
  - Apply hair style from prompt/reference image
  - Hair mesh should have spring bones (UniRig predicts these)

- Write end-to-end test:
  - Input: "A fantasy elf ranger with green eyes and braided hair"
  - Output: Textured template mesh with correct UVs and materials
```

Deliverable: Text/image → textured template mesh with appropriate materials.

Phase 3: Pure AI Generation Path (2–3 weeks)

Goal: Support fully AI-generated meshes as an alternative to templates.

```
Tasks for Claude Code:
- Integrate TRELLIS.2 (or Hunyuan3D) as mesh generator:
  - Image → raw mesh pipeline
  - Handle PBR material extraction

- Build automatic retopology pipeline:
  - Input: high-poly AI mesh (100k+ triangles)
  - Use Instant Meshes (CLI) or Blender's Quadriflow
  - Target: 8000-10000 faces
  - Preserve face region density (more polys around eyes/mouth)

- Build automatic face component separation:
  - Use MediaPipe face landmarks on rendered views of the mesh
  - Project 2D landmarks → 3D vertex positions
  - Identify eye region, mouth region, forehead, chin
  - Separate by vertex proximity to landmark clusters
  - Model teeth/tongue from prefab library, scaled to fit
  - Position eye spheres based on detected eye socket geometry

- Integrate deformation transfer for non-template meshes:
  - NRICP (Non-Rigid ICP) to fit ICT FaceKit reference → target face
  - Transfer all 52 ARKit shapes
  - Validate deformation quality

- Integrate UniRig for novel body shapes:
  - Run skeleton prediction on retopologized mesh
  - Run skinning weight prediction
  - Remap bone names to Mixamo convention

- Quality validation:
  - Automated checks: mesh is manifold, no inverted normals,
    blend shapes have reasonable vertex displacements,
    skeleton has expected bone count, skinning weights sum to 1.0
```

Deliverable: Image → complete rigged mesh with blend shapes (lower quality than template path but fully novel).

Phase 4: Animation & Export Pipeline (1–2 weeks)

Goal: Attach animations and export TalkingHead-compatible GLB.

```
Tasks for Claude Code:
- Build animation attachment system:
  - Library of Mixamo FBX animations (idle, walk, run, wave, etc.)
  - Retarget animations to character skeleton via Blender
  - Handle bone name mapping (Mixamo → template → export)

- GLB export pipeline:
  - Include all morph targets with correct naming:
    ARKit: eyeBlinkLeft, eyeBlinkRight, jawOpen, mouthSmile, etc.
    Oculus: viseme_aa, viseme_E, viseme_I, etc.
    Extra: mouthOpen, mouthSmile, eyesClosed, eyesLookUp, eyesLookDown
  - Embed animations or export as separate FBX files
  - LOD generation: export at 10k, 5k, 3k face counts
  - Draco mesh compression option

- TalkingHead integration test:
  - Load generated GLB into TalkingHead test app
  - Verify lip-sync works with HeadTTS
  - Verify body animations play correctly
  - Verify facial expressions respond to emoji input
  - Test with HeadAudio for audio-driven visemes

- Build quality scoring system:
  - Score blend shape deformation quality (0-1)
  - Score rig deformation under animation
  - Score texture quality
  - Flag any issues for human review
```

Deliverable: Complete pipeline produces TalkingHead-ready GLB files that pass automated validation.

Phase 5: Web Service & API (2-3 weeks)

Goal: Wrap the pipeline in a web application.

```
Tasks for Claude Code:
- Backend API (Python FastAPI or similar):
  - POST /generate — accepts prompt or image, returns job ID
  - GET /status/{job_id} — returns progress
  - GET /download/{job_id} — returns GLB file
  - POST /customize — adjust parameters of existing generation
  - WebSocket for real-time progress updates

- Job queue system:
  - Redis + Celery (or similar) for async job processing
  - GPU worker pool management
  - Job priority, cancellation, retry

- Frontend (React or similar):
  - Text prompt input with style selector
  - Image upload with drag-and-drop
  - Real-time preview (Three.js viewer showing generation progress)
  - Customization sliders (body proportions, face shape)
  - Download options (GLB, FBX, separate animations)
  - TalkingHead live preview (embed TalkingHead to test avatar)

- User accounts & gallery:
  - Save/share generated avatars
  - Public gallery of user creations
  - Generation history

- Containerization:
  - Docker compose for full stack
  - GPU container with all ML models pre-loaded
  - Blender headless in the pipeline container
  - Separate containers for web, queue, workers, database
```

Deliverable: Working web application that anyone can use.

Phase 6: Optimization & Scale (Ongoing)

```
- Model quantization (FP8/INT8 for TRELLIS.2) to reduce VRAM
- Caching: pre-generate template variations, cache blend shapes
- Batch processing: generate multiple avatars per GPU cycle
- CDN for serving generated GLB files
- A/B testing different generation models
- User feedback loop to improve quality
```

```
- Fine-tune TRELLIS.2 on humanoid character dataset
- Monitoring, logging, error handling
```

Feasibility Assessment

What's Proven and Works Today

Component	Status	Confidence
Text → concept images	Production-ready (Stable Diffusion, FLUX)	Very High
Image → 3D mesh	Production-ready (TRELLIS.2, Hunyuan3D)	High
Auto-rigging (skeleton + skinning)	Production-ready (UniRig)	High
Deformation transfer (ARKit shapes)	Research-proven, open source	Medium-High
Blender headless automation	Battle-tested, widely used	Very High
GLB export with morph targets	Built into Blender	Very High
TalkingHead integration	Well-documented, working examples	High

What's the Hardest Part

1. **Automatic face component separation on AI-generated meshes** — This is the single riskiest stage. Identifying and separating eye sockets, teeth, and tongue from a fused AI mesh is not a solved problem. The template approach sidesteps this entirely. For the pure AI path, you'd need a combination of MediaPipe landmarks + heuristic geometry analysis + prefab component insertion.
2. **Blend shape quality on diverse face topologies** — Deformation transfer works well when source and target faces are similar in proportions. For extreme stylization (very large anime eyes, very wide cartoon faces), the transferred blend shapes may need manual tweaking. Confidence is moderate for realistic styles, lower for highly stylized.
3. **Consistent texture quality** — AI texture generation on existing meshes (Hunyuan3D-Paint style) is still imperfect, especially around UV seams and face details. Getting consistently good textures from text prompts alone is an active research area.

Realistic Timeline

Phase	Duration	Cumulative
Phase 0: Foundation	1–2 weeks	2 weeks
Phase 1: Templates	2–3 weeks	5 weeks
Phase 2: AI Appearance	2–3 weeks	8 weeks
Phase 3: Pure AI Path	2–3 weeks	11 weeks
Phase 4: Animation/Export	1–2 weeks	13 weeks
Phase 5: Web Service	2–3 weeks	16 weeks
Total to MVP	**~4 months**	

This assumes one developer working with Claude Code full-time. With Claude Code handling boilerplate, API integration, testing, and Blender scripting, the timeline is compressed significantly versus traditional development.

Hardware for Development

Your current machine (RTX 3080 16GB) can develop most of this:

- Template creation in Blender: excellent
- Blender headless scripting: excellent
- Deformation transfer: runs on CPU, fine
- UniRig inference: fits in 16GB
- Hunyuan3D-2mini (turbo): fits in 16GB for shape
- TRELLIS.2: needs 24GB — use cloud GPU for this step

For production serving, you'd need:

- One GPU server with RTX 4090 (24GB) or RTX 5090 (32GB): ~$1,500–3,000
- Or cloud GPU instances on demand: ~$0.50–2.00/hour
- Standard web server for API/frontend: ~$50–100/month

Hardware for Production Service

Scale	Setup	Monthly Cost
Hobby/Demo	Your RTX 3080 + cloud GPU bursts	~$50–100
Small (50 avatars/day)	1x RTX 4090 server + web VPS	~$200–400
Medium (500/day)	2x RTX 5090 + load balancer	~$500–1,000
Large (5000/day)	Cloud GPU cluster (RunPod/Vast.ai)	~$2,000–5,000

Key Open Source Components Summary

Component	Repo	License	Purpose
TRELLIS.2	microsoft/TRELLIS.2	MIT	Image → 3D mesh generation
Hunyuan3D 2.1	Tencent-Hunyuan/Hunyuan3D-2	Tencent	Text/Image → 3D with textures
UniRig	VAST-AI-Research/UniRig	MIT	Automatic skeleton + skinning
Deformation Transfer	vasiliskatr/deformation_transfer_ARkit_blendshapes	Open	ARKit blend shape generation
ICT FaceKit	ICT-VGL/ICT-FaceKit	Free/Academic	Reference face with blend shapes
Blender	blender.org	GPL	Headless mesh processing engine

Component	Repo	License	Purpose
Instant Meshes	wjakob/instant-meshes	BSD	Automatic retopology
TalkingHead	met4citizen/TalkingHead	MIT	Target platform / validation
HeadTTS	met4citizen/HeadTTS	MIT	TTS with viseme output
HeadAudio	met4citizen/HeadAudio	MIT	Audio-driven visemes
Stable Diffusion / FLUX	Various	Various	Concept image generation
ComfyUI	comfyanonymous/ComfyUI	GPL	Optional workflow orchestration

How Claude Code Specifically Helps

Claude Code is exceptionally well-suited for this project because:

1. **Blender Python scripting** — Claude Code can write and iterate on bpy scripts rapidly, which is the glue connecting every pipeline stage.
2. **API integration** — Connecting TRELLIS.2, UniRig, Hunyuan3D, and Blender involves Python SDK/CLI integration that Claude Code handles fluently.
3. **3D math** — Deformation transfer, vertex correspondence, skinning weight computation — all involve linear algebra that Claude Code can implement and debug.
4. **Web service scaffolding** — FastAPI backend, React frontend, Docker containers, queue workers — all standard patterns Claude Code generates quickly.
5. **Testing and validation** — Writing automated tests that verify mesh integrity, blend shape naming, skeleton structure, GLB compliance.
6. **Iterative development** — The pipeline has many stages that need independent development and testing. Claude Code's ability to work on one module while keeping context of the whole system is valuable.

Bottom Line: Is This Feasible?

Yes, with caveats.

The template-hybrid path (Phase 1 + 2 + 4 + 5) is highly feasible and could produce a working MVP in ~2 months. It would generate unique-looking avatars from text/images that work perfectly in TalkingHead, because the underlying mesh structure is guaranteed correct by the template system.

The pure AI generation path (Phase 3) adds another month and introduces significant technical risk around face component separation and blend shape quality. It's worth pursuing but should be Phase 2, not Phase 1.

The market timing is excellent: Ready Player Me just died, 8,000+ developers need an alternative, and the open source tools to build this only became available in the last 6–12 months. Nobody has assembled them into a unified service yet.